

Capture the Flag 3: Data-Driven UI

Goal: Separate application data from the UI and handle multiple pieces of data in a usable interface.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- **Explain** an error (not **fix** your error)
- Clarify documentation

You may **NOT** ask AI to generate the solution/code for a flag.

Flag 0 — Create Your Feature Branch

Objective: Create a feature branch before modifying your application.

Create:

`feature/data-ui`

Screenshot:

```
Windows PowerShell
- d, --[no-]detach      detach HEAD at named commit
- t, --[no-]track[=(direct|inherit)]
                        set branch tracking configuration
- f, --[no-]force       force checkout (throw away local modifications)
--[no-]orphan <new-branch>
                        new unborn branch
--[no-]overwrite-ignore
                        update ignored files (default)
--[no-]ignore-other-worktrees
                        do not check if another worktree is using this branch
- 2, --ours            checkout our version for unmerged files
- 3, --theirs          checkout their version for unmerged files
- p, --[no-]patch       select hunks interactively
- U, --unified <n>     generate diffs with <n> lines context
--inter-hunk-context <n>
                        show context between diff hunks up to the specified number of lines
--[no-]ignore-skip-worktree-bits
                        do not limit pathspecs to sparse entries only
--[no-]pathspec-from-file <file>
                        read pathspec from file
--[no-]pathspec-file-nul
                        with --pathspec-from-file, pathspec elements are separated with NUL character

PS C:\Users\kevin\IT314\IT314AB_Villacorte> git checkout feature/data-ui
branch 'feature/data-ui' set up to track 'origin/feature/data-ui'.
Switched to a new branch 'feature/data-ui'
PS C:\Users\kevin\IT314\IT314AB_Villacorte> git branch
* feature/data-ui
  main
PS C:\Users\kevin\IT314\IT314AB_Villacorte> |
```

- Terminal showing your current branch

Flag 1 — Identify the Data

Objective: Identify the appropriate Dart data type for the information in your profile.

Research basic Dart data types and identify at least **four different types** that could be used in your application.

Examples:

String
int
double
bool

Create a table identifying the information and its appropriate data type.

Screenshot:

NAME	STRING
COURSE	STRING
AGE	INT
HEIGHT	DOUBLE
STATUS	BOOL
HOBBY	STRING

- Your identified data
- The corresponding data types

Flag 2 — Create Your Data

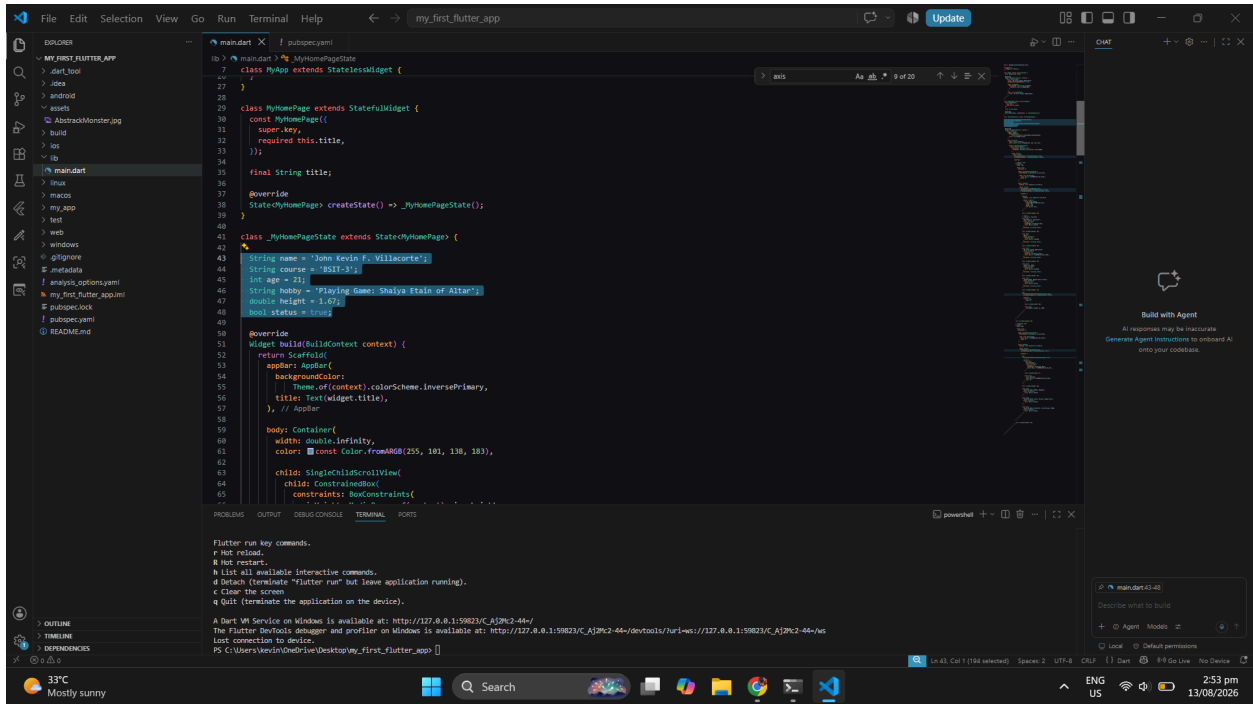
Objective: Create appropriately typed variables for your profile.

Create variables for at least:

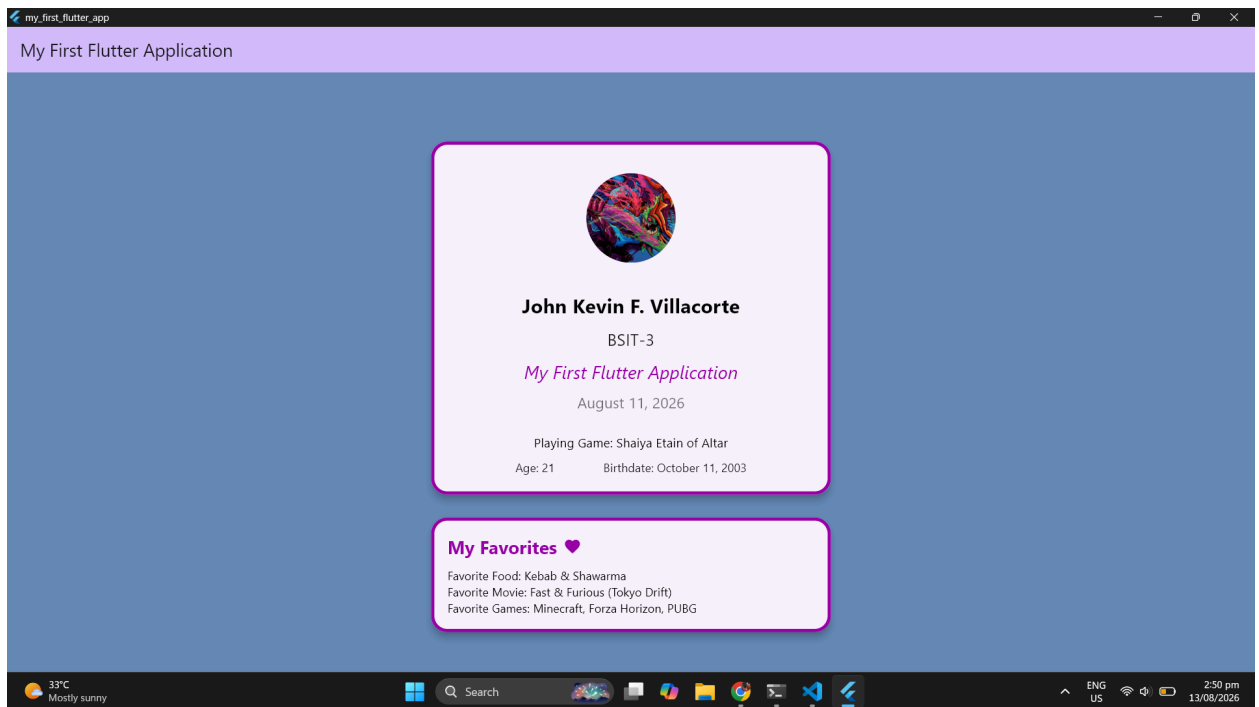
- Name
- Course & Section
- Age
- Hobby
- Height
- Student status

Your data should exist **separately from your UI widgets**.

Screenshot:



- Your variables
- Their data types



- Application still running



Flag 3 — Connect Data to UI

Objective: Replace your hardcoded information with your variables.

Your profile should now display information using the variables you created.

Test:

Change the value of one of your variables.

The UI should automatically display the new value.

Screenshot:

```

117     Text(
118       name,
119       style: TextStyle(
120         fontSize: 24,
121         fontWeight: FontWeight.bold,
122         color: Colors.black,
123       ), // TextStyle
124       textAlign: TextAlign.center,
125     ), // Text
126
127     const SizedBox(height: 10),
128

```

```

129     Text(
130       course,
131       style: TextStyle(
132         fontSize: 20,
133         color: Colors.black87,
134       ), // TextStyle
135       textAlign: TextAlign.center,
136     ), // Text

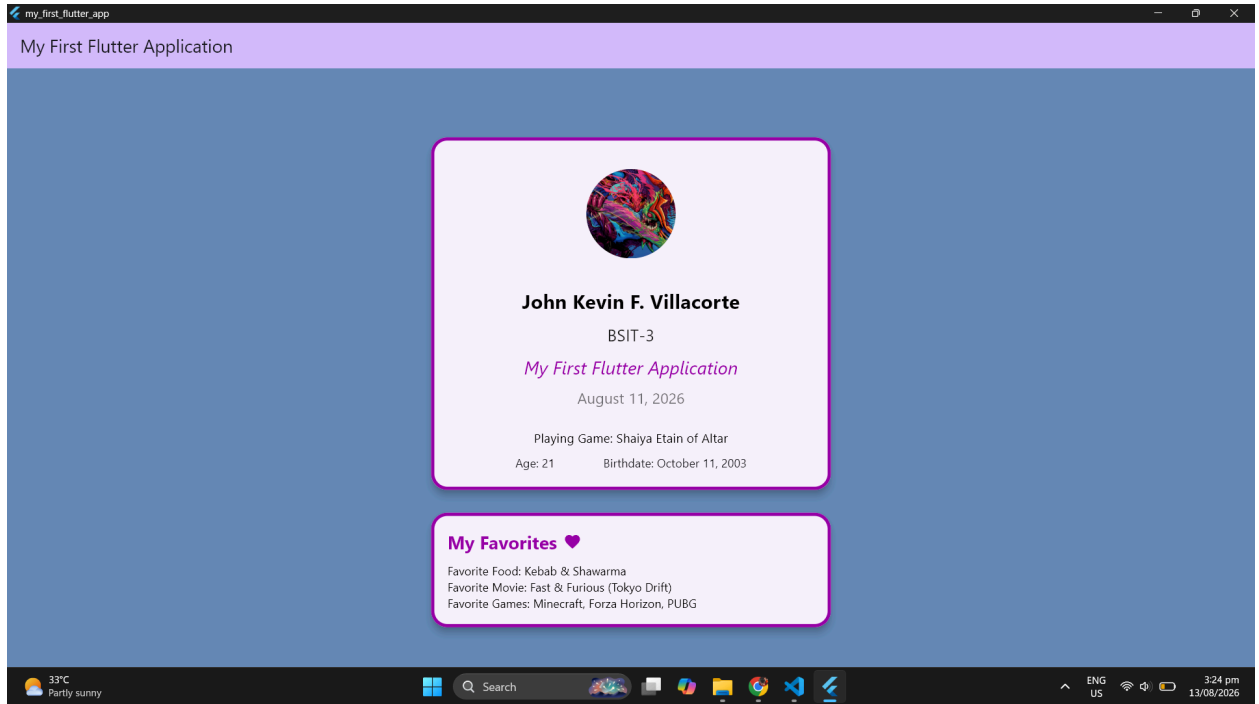
```

```

163     Text(
164       hobby,
165       style: TextStyle(
166         fontSize: 15,
167         color: Colors.black,
168       ), // TextStyle
169       textAlign: TextAlign.center,
170     ), // Text
171
172     const SizedBox(height: 10),
173
174     Row(
175       mainAxisAlignment: MainAxisAlignment.center,
176       crossAxisAlignment: CrossAxisAlignment.center,
177
178       children: [
179         Text(
180           'Age: $age',
181         ), // Text

```

- Variables
- Widgets displaying the variables



- Updated application

🚩 Flag 4 — Make the Image Data-Driven0

Objective: Make your profile image part of your data.

Create a variable containing the path to your profile image.

Use the variable when displaying the image.

Then change the variable to another registered image.

The image displayed by the application should change.

Screenshot:

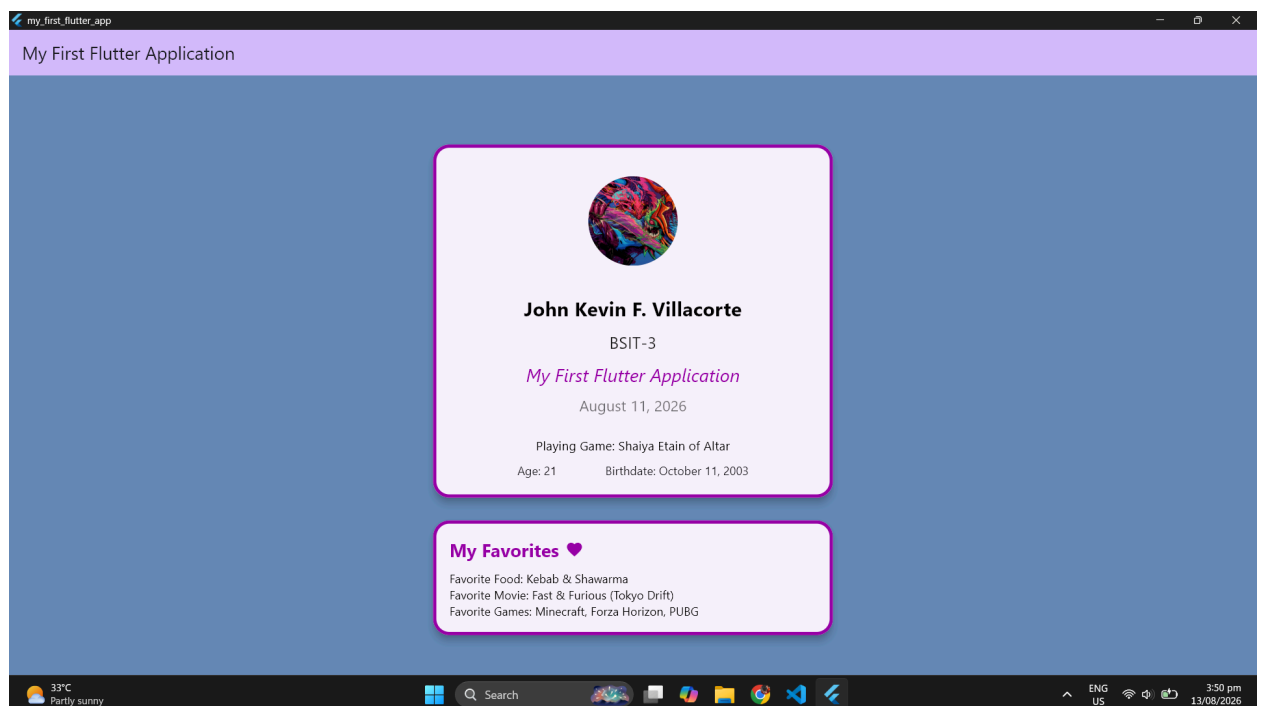
```

49      String imagePath = 'assets/AbstrackMonster.jpg';

106      child: ClipOval(
107        child: Image.asset(
108          imagePath,
109          width: 110,
110          height: 110,
111          fit: BoxFit.cover,
112        ), // Image.asset
113      ), // ClipOval

```

- Image variable
- Image widget using the variable



- Updated application

🚩 Flag 5 — Five Profiles, One Design

Objective: Create at least **five different profile cards** using the same visual design.

Each profile must contain at least:

- Image
- Name
- Course & Section
- Age
- Hobby

The five profiles must contain different information.

Important

All five profiles should use the **same visual design**.

You are demonstrating that the same UI can represent different data.

Screenshot:

```
const Text(  
  'Recmar Atis',  
  style: TextStyle(  
    fontSize: 24,  
    fontWeight: FontWeight.bold,  
    color: Colors.black,  
  ), // TextStyle  
  textAlign: TextAlign.center,  
), // Text  
  
const SizedBox(height: 10),  
  
const Text(  
  'BSTI-2',  
  style: TextStyle(  
    fontSize: 20,  
    color: Colors.black87,  
  ), // TextStyle  
  textAlign: TextAlign.center,  
), // Text  
  
const SizedBox(height: 10),  
  
const Text(  
  'August 07, 2026',  
  style: TextStyle(  
    fontSize: 18,  
    color: Colors.black54,  
  ), // TextStyle  
  textAlign: TextAlign.center,  
), // Text  
  
const SizedBox(height: 25),  
  
const Text(  
  'Playing Basketball',  
  style: TextStyle(  
    fontSize: 15,  
    color: Colors.black,  
  ), // TextStyle  
  textAlign: TextAlign.center,  
), // Text
```

```
const Text(  
  'Larfie Atis',  
  style: TextStyle(  
    fontSize: 24,  
    fontWeight: FontWeight.bold,  
    color: Colors.black,  
  ), // TextStyle  
  textAlign: TextAlign.center,  
), // Text  
  
const SizedBox(height: 10),  
  
const Text(  
  'BSIT-3',  
  style: TextStyle(  
    fontSize: 20,  
    color: Colors.black87,  
  ), // TextStyle  
  textAlign: TextAlign.center,  
), // Text  
  
const SizedBox(height: 10),  
  
const Text(  
  'August 13, 2026',  
  style: TextStyle(  
    fontSize: 18,  
    color: Colors.black54,  
  ), // TextStyle  
  textAlign: TextAlign.center,  
), // Text  
  
const SizedBox(height: 25),  
  
const Text(  
  'Playing Basketball',  
  style: TextStyle(  
    fontSize: 15,  
    color: Colors.black,  
  ), // TextStyle  
  textAlign: TextAlign.center,  
), // Text
```

```
const Text(  
  'Charlie Atis',  
  style: TextStyle(  
    fontSize: 24,  
    fontWeight: FontWeight.bold,  
    color: Colors.black,  
  ), // TextStyle  
  textAlign: TextAlign.center,  
), // Text  
  
const SizedBox(height: 10),  
  
const Text(  
  'BSIT-3',  
  style: TextStyle(  
    fontSize: 20,  
    color: Colors.black87,  
  ), // TextStyle  
  textAlign: TextAlign.center,  
), // Text  
  
const SizedBox(height: 10),  
  
const Text(  
  'August 13, 2026',  
  style: TextStyle(  
    fontSize: 18,  
    color: Colors.black54,  
  ), // TextStyle  
  textAlign: TextAlign.center,  
), // Text  
  
const SizedBox(height: 25),  
  
const Text(  
  'Playing Basketball',  
  style: TextStyle(  
    fontSize: 15,  
    color: Colors.black,  
  ), // TextStyle  
  textAlign: TextAlign.center,  
), // Text
```

```
const Text(  
  'Clifford Atis',  
  style: TextStyle(  
    fontSize: 24,  
    fontWeight: FontWeight.bold,  
    color: Colors.black,  
  ), // TextStyle  
  textAlign: TextAlign.center,  
), // Text  
  
const SizedBox(height: 10),  
  
const Text(  
  'BSIT-3',  
  style: TextStyle(  
    fontSize: 20,  
    color: Colors.black87,  
  ), // TextStyle  
  textAlign: TextAlign.center,  
), // Text  
  
const SizedBox(height: 10),  
  
const Text(  
  'August 13, 2026',  
  style: TextStyle(  
    fontSize: 18,  
    color: Colors.black54,  
  ), // TextStyle  
  textAlign: TextAlign.center,  
), // Text  
  
const SizedBox(height: 25),  
  
const Text(  
  'Watching SpiderMan',  
  style: TextStyle(  
    fontSize: 15,  
    color: Colors.black,  
  ), // TextStyle  
  textAlign: TextAlign.center,  
), // Text
```

```

const Text(
  'Vicente Ramon',
  style: TextStyle(
    fontSize: 24,
    fontWeight: FontWeight.bold,
    color: Colors.black,
  ), // TextStyle
  textAlign: TextAlign.center,
), // Text

const SizedBox(height: 10),

const Text(
  'BSIT-3',
  style: TextStyle(
    fontSize: 20,
    color: Colors.black87,
  ), // TextStyle
  textAlign: TextAlign.center,
), // Text

const SizedBox(height: 10),

const Text(
  'August 13, 2026',
  style: TextStyle(
    fontSize: 18,
    color: Colors.black54,
  ), // TextStyle
  textAlign: TextAlign.center,
), // Text

const SizedBox(height: 25),

const Text(
  'Playing Guitar',
  style: TextStyle(
    fontSize: 15,
    color: Colors.black,
  ), // TextStyle
  textAlign: TextAlign.center,
), // Text

```

- Your 5 different profile data



Flag 6 — THE HIDDEN PROBLEM

You didn't see your UI?

Objective: Make all five profiles accessible to the user.

You just added five profile cards.

Now run your application.

Something is wrong.

Look carefully at your emulator.

Can you see all five profiles?

Maybe you can see the first one...

Maybe part of the second...

But where are the rest?

That's right.

You can't see them.

Your UI is larger than the available screen.

The profiles exist.

The data exists.

The widgets exist.

But the user can't access everything.

Your Mission:

Figure out how Flutter allows users to **scroll through content that is larger than the available screen**.

Research Flutter's **scrolling widgets** and determine which one is appropriate for your current layout.

You must make it possible to scroll through all five profiles.

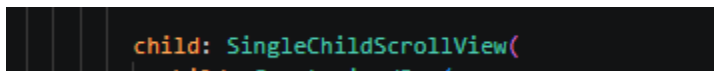
Requirement:

The user must be able to:

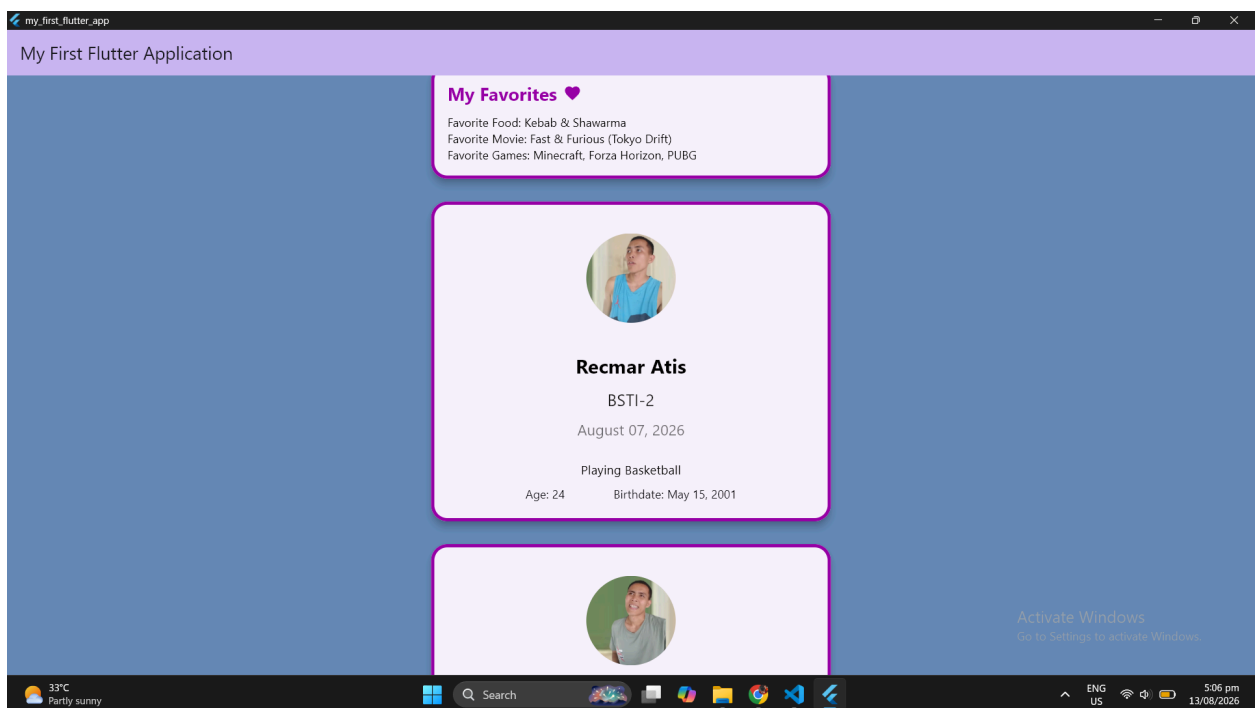
Scroll → see Profile 1 → Profile 2 → Profile 3 → Profile 4 → Profile 5

without changing the size of the screen.

Screenshot:



- The scrolling widget in your code
- Top portion of the application



- The rest of the portion of the application showing the later profiles

Flag 7 — Survive the Database

Objective: Prepare your UI for incomplete data.

Now imagine your profiles aren't manually created.

Someday, the data will come from a database.

And real data isn't always perfect.

Some information may be:

- Missing
- Empty
- `null`
- Not provided

Create at least **three profiles with missing information**.

For example:

Profile A

Name: Maria

Course: BSIT

Hobby: **missing**

Profile B

Name: Juan

Course: **missing**

Hobby: Gaming

Profile C

Name: **missing**

Course: MMA

Hobby: Drawing

Your application must **not crash** when information is missing.

Instead, provide an appropriate fallback.

Examples:

Hobby: Not provided

Course: Unknown

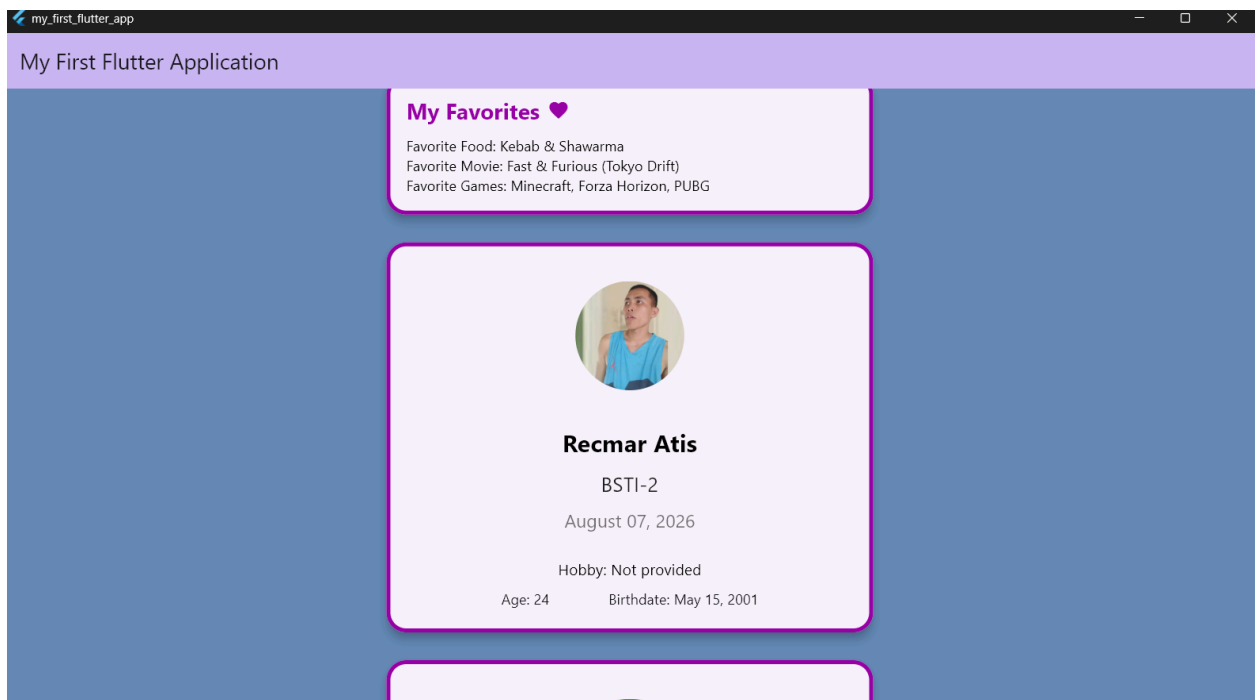
Challenge

Your placeholders should look intentional and clean.

Screenshot:

```
42  
43   String name = 'John Kevin F. Villacorte';  
44   String course = 'BSIT-3';  
45   int age = 21;  
46   String hobby = 'Playing Game: Shaiya Etain of Altar';  
47   String imagePath = 'assets/AbstrackMonster.jpg';  
48  
49   String? name2 = 'Recmar Atis';  
50   String? course2 = 'BSTI-2';  
51   String? hobby2;  
52  
53   String? name3 = 'Larfie Atis';  
54   String? course3;  
55   String? hobby3 = 'Playing Basketball';  
56  
57   String? name4;  
58   String? course4 = 'BSIT-3';  
59   String? hobby4 = 'Playing Basketball';  
60
```

- Incomplete profile data



- Application displaying fallback values

- Application still functioning correctly

Flag 8 — Save Your Progress

Objective: Commit and push your completed Data-Driven UI CTF.

Use an appropriate commit message following proper Git standards.

Branch:

`feature/data-ui`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit